ÁRBOLES RECURSIVOS

Inserción, modificación, eliminación y recorridos · SJA-UTA

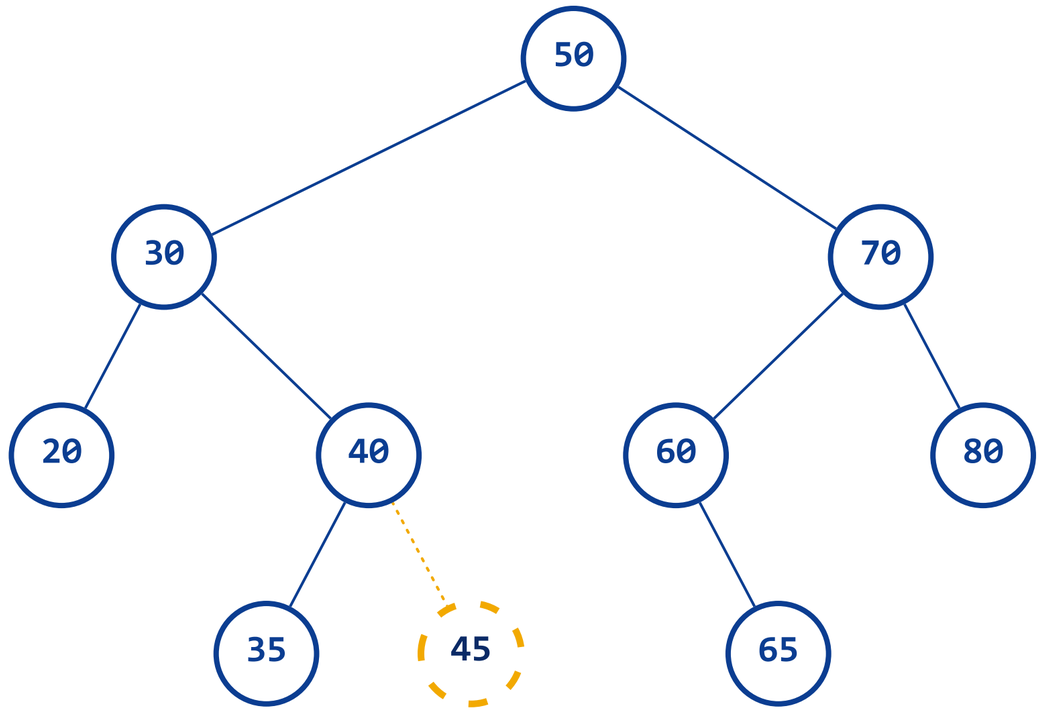
Grupo 2 · Estructura de Datos · UTA · APE 3

RECORRIDO VIVIENTE

Llega la matrícula 45 al paralelo SW-N3-B.

¿Dónde la ubicamos?

- Cada voluntario es un nodo y lleva una tarjeta con su matrícula.
- Entran en el orden de inserción: 50, 30, 70, 20, 40, 60, 80, 35, 65.
- La matrícula 45 baja comparando, nodo por nodo.
- El lugar vacío donde se detiene es el caso base.

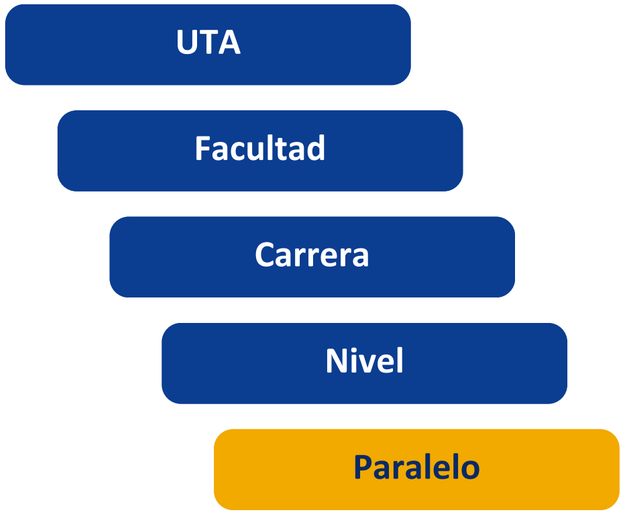


El círculo punteado es el lugar libre para el 45

Expone: Kenneth

CONTEXTO SJA-UTA

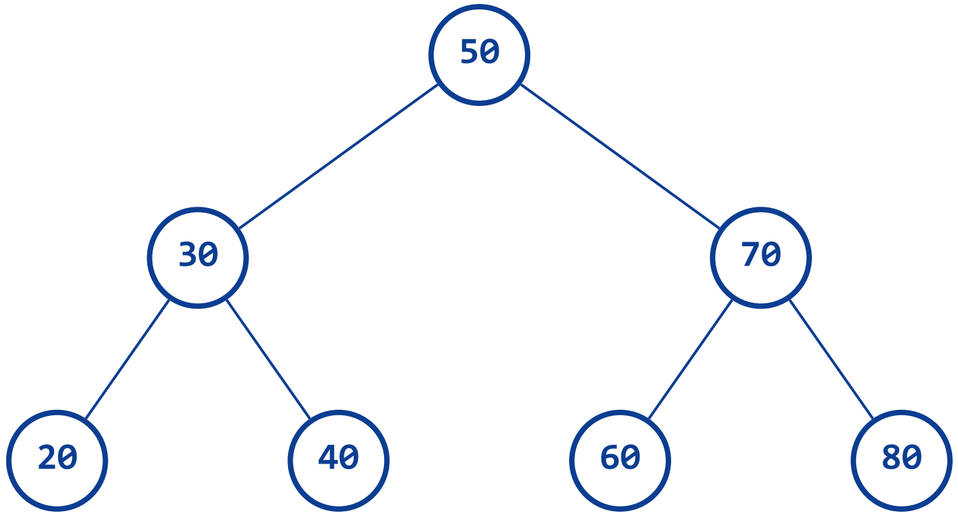
ÁRBOL GENERAL · 88 unidades



Un nodo puede tener varios hijos

Cada paralelo guarda sus estudiantes en un ABB

ABB POR PARALELO · clave: matrícula



NodoUnidad
`dato · hijos (lista, varios)`

NodoEstudiante
`dato · izquierdo · derecho (máx. 2)`

Expone: Damian

RECURSIVIDAD EN ÁRBOLES

1

CASO BASE

nodo == null: no hay nada que hacer, se regresa.

2

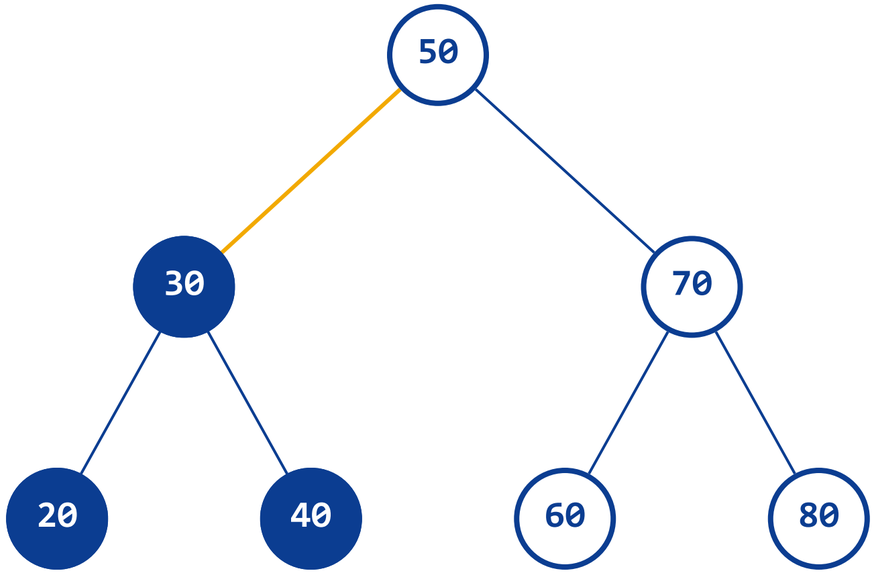
MI PARTE

Se procesa el nodo actual (comparar, visitar, enlazar).

3

CASO RECURSIVO

La misma función en cada hijo: un subárbol más pequeño.



Cada hijo es la raíz de un árbol más pequeño. El de la izquierda (en azul) se resuelve igual que el árbol completo.

Expone: Darwin

BUSCAR: ÁRBOL GENERAL VS ABB

Árbol general · ArbolOrganizacional

- Revisa el nodo; si no es, prueba con cada hijo.
- El código se normaliza: trim() y mayúsculas.
- Puede visitar las 88 unidades: O(n).

```
for (NodoUnidad hijo : nodo.getHijos()) {
    NodoUnidad encontrado = buscar(hijo, codigo);
    if (encontrado != null) { return encontrado; }
}
```

ABB · ArbolEstudiantes

- Compara: menor va a la izquierda, mayor a la derecha.
- Descarta un lado completo en cada paso.
- Costo O(h): con altura 3 son máximo 4 comparaciones.

```
return matricula < actual
    ? buscar(nodo.getIzquierdo(), matricula)
    : buscar(nodo.getDerecho(), matricula);
```

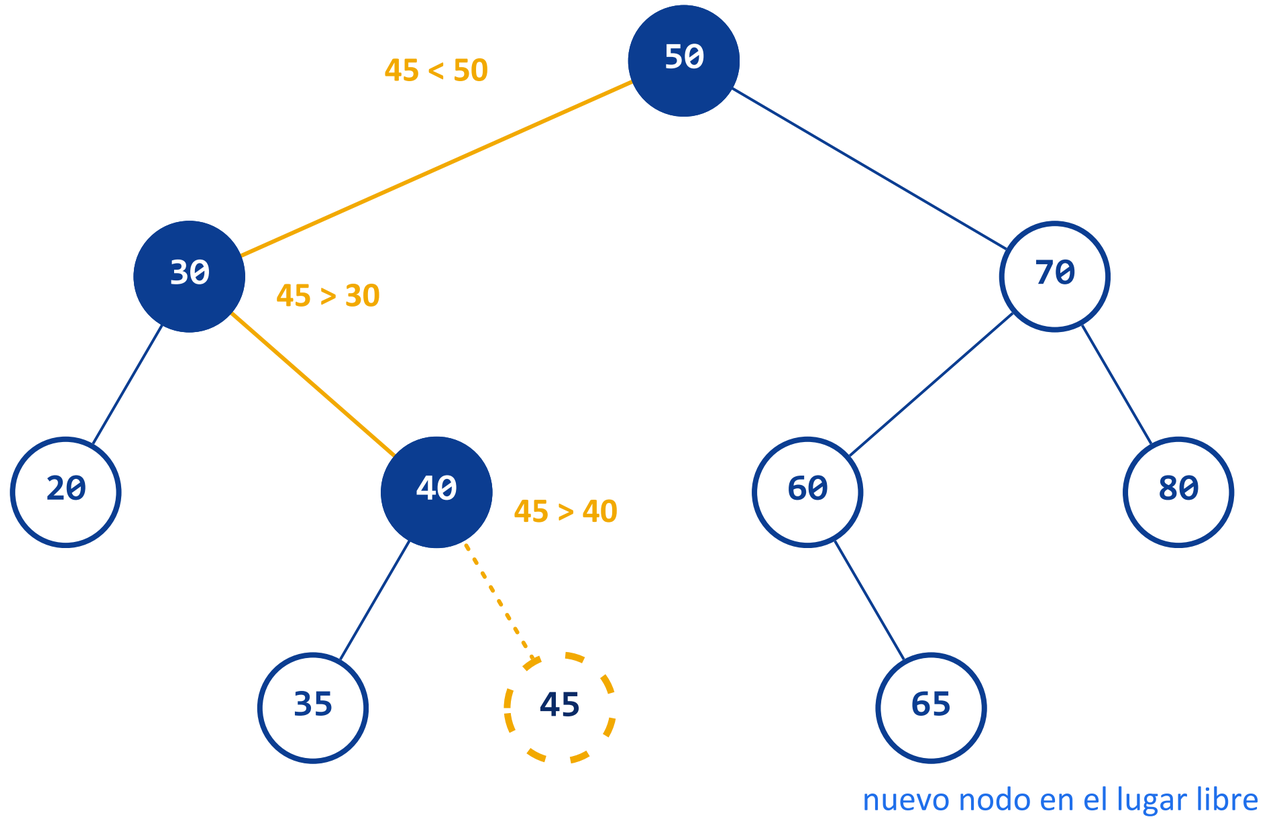
Mismo problema, distinta estructura: el orden del ABB evita recorrerlo todo.

Expone: Damian

INSERCIÓN: EL CAMINO DEL 45

45 < 50 va a la izquierda
45 > 30 va a la derecha
45 > 40 va a la derecha
Lugar vacío (null): caso base, se crea el nodo
Si la matrícula ya existe: se rechaza

Costo O(h)



Expone: Kenneth

INSERTAR: ITERATIVA VS RECURSIVA

	Iterativa (while)	Recursiva (caso base)
Cómo baja	Un puntero actual y otro padre	Cada llamada baja un nivel
Enlazar el nodo nuevo	if (padre.izq == null) padre.izq = nuevo	setIzquierdo(insertar(...)) enlaza al regresar
Memoria extra	O(1)	O(h): pila de llamadas
Riesgo	Olvidar actualizar al padre	Sin caso base: desbordamiento de pila
Tiempo	O(h)	O(h)

Mismo resultado y mismo O(h). Usamos la recursiva porque copia la definición del árbol: un nodo y dos subárboles.

Expone: Kenneth

INSERCIÓN: CÓDIGO

ArbolEstudiantes.java

```java
public boolean insertar(Estudiante estudiante) {
    if (buscar(estudiante.getMatricula()) != null) {
        return false; // matricula repetida
    }
    raiz = insertar(raiz, estudiante);
    return true;
}

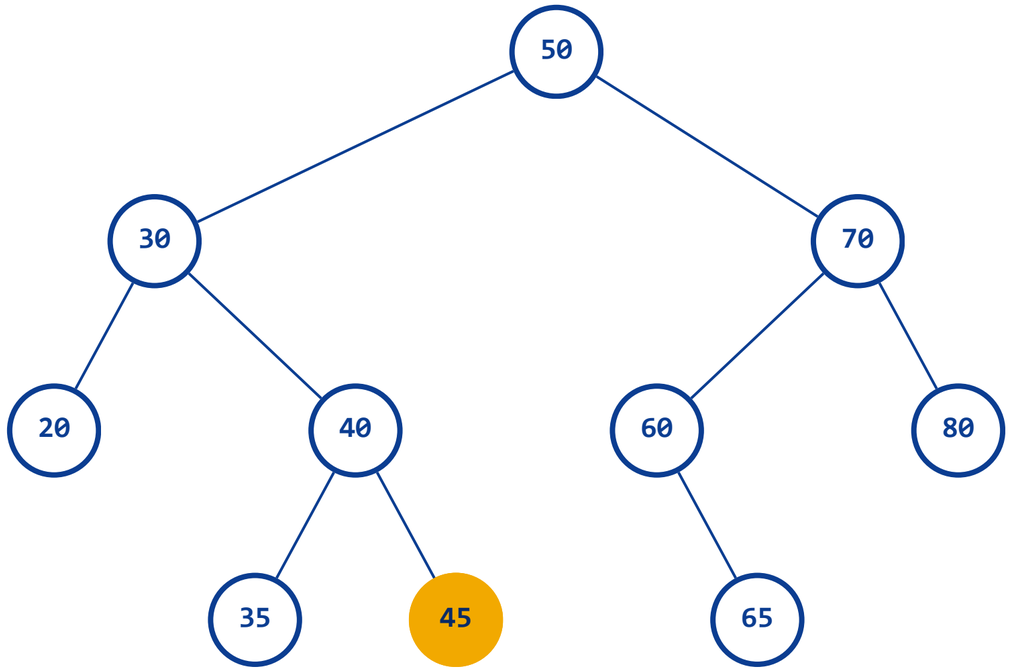
private NodoEstudiante insertar(NodoEstudiante nodo, Estudiante estudiante) {
    if (nodo == null) {
        return new NodoEstudiante(estudiante); // caso base: lugar libre
    }
    if (estudiante.getMatricula() < nodo.getDato().getMatricula()) {
        nodo.setIzquierdo(insertar(nodo.getIzquierdo(), estudiante));
    } else {
        nodo.setDerecho(insertar(nodo.getDerecho(), estudiante));
    }
    return nodo;
}
```

QUÉ MIRAR

- Caso base: lugar libre, se crea el nodo.
- Se baja por el lado que indica la comparación.
- return nodo enlaza al regresar.
- El método público rechaza repetidas.

Expone: Kenneth

LOS TRES RECORRIDOS



PREORDEN raíz · izq · der

50 30 20 40 35 45 70 60 65 80

INORDEN izq · raíz · der

20 30 35 40 45 50 60 65 70 80

POSORDEN izq · der · raíz

20 35 45 40 30 65 60 80 70 50

En un ABB el inorden sale ordenado por matrícula. Mismo árbol, mismo O(n): solo cambia cuándo se visita el nodo.

Expone: Darwin

¿DÓNDE VA LA VISITA?

PREORDEN

1. **VISITA**
2. izquierdo
3. derecho

Copiar el árbol o imprimir la jerarquía: primero el padre.

INORDEN

1. izquierdo
2. **VISITA**
3. derecho

Lista ordenada: matrículas de menor a mayor.

POSORDEN

1. izquierdo
2. derecho
3. **VISITA**

Borrar el árbol: primero los hijos, al final el padre.

El caso base y las llamadas recursivas son iguales en los tres. Solo cambia la línea de la visita.

Expone: Darwin

RECORRIDOS: CÓDIGO

ArbolEstudiantes.java

```java
private void preorden(NodoEstudiante nodo, List<Estudiante> lista) {
    if (nodo == null) return;              // caso base
    lista.add(nodo.getDato());             // VISITA
    preorden(nodo.getIzquierdo(), lista);
    preorden(nodo.getDerecho(), lista);
}
private void inorden(NodoEstudiante nodo, List<Estudiante> lista) {
    if (nodo == null) return;              // caso base
    inorden(nodo.getIzquierdo(), lista);
    lista.add(nodo.getDato());             // VISITA
    inorden(nodo.getDerecho(), lista);
}
private void posorden(NodoEstudiante nodo, List<Estudiante> lista) {
    if (nodo == null) return;              // caso base
    posorden(nodo.getIzquierdo(), lista);
    posorden(nodo.getDerecho(), lista);
    lista.add(nodo.getDato());             // VISITA
}
```

QUÉ MIRAR

- El caso base es idéntico en los tres.
- Solo cambia dónde está lista.add.
- Cada nodo se visita una vez: O(n).
- Árbol vacío: lista vacía.

Expone: Darwin

LA TRAZA ES LA PILA DE LLAMADAS

ArbolEstudiantes.java · trazaRecorrido

```java
private void traza(NodoEstudiante nodo, TipoRecorrido tipo, int nivel,
                   StringBuilder traza, StringBuilder secuencia) {
    String sangria = "  ".repeat(nivel);
    if (nodo == null) {
        traza.append(sangria).append("null -> regresa\n");
        return;                                  // caso base
    }
    int m = nodo.getDato().getMatricula();
    traza.append(sangria).append("entra ").append(m).append("\n");
    if (tipo == TipoRecorrido.PREORDEN) visitar(m, sangria, traza, secuencia);
    traza(nodo.getIzquierdo(), tipo, nivel + 1, traza, secuencia);
    if (tipo == TipoRecorrido.INORDEN) visitar(m, sangria, traza, secuencia);
    traza(nodo.getDerecho(), tipo, nivel + 1, traza, secuencia);
    if (tipo == TipoRecorrido.POSORDEN) visitar(m, sangria, traza, secuencia);
}
```

Salida real · árbol 50, 30, 70

```
entra 50
  entra 30
    null -> regresa
  VISITA 30
    null -> regresa
VISITA 50
  entra 70
    null -> regresa
  VISITA 70
    null -> regresa
Secuencia INORDEN: 30 50 70
```

Cada nivel de sangría es una llamada pendiente en la pila. Se regresa con "null -> regresa" y se sigue con el otro hijo.

Expone: Darwin

MODIFICACIÓN

SE PUEDE CAMBIAR

Cédula, nombres y promedio.

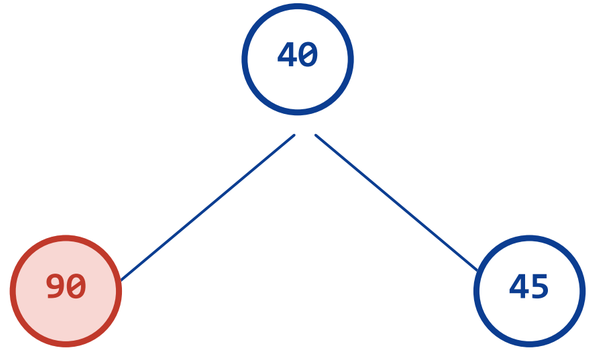
NO SE CAMBIA

La matrícula: es la clave que ordena el ABB.

VALIDAR ANTES

new Estudiante(...) lanza excepción y el dato real no se toca.

Si la clave cambiara…



Si 35 pasara a ser 90, quedaría un 90 a la izquierda del 40: el ABB ya no está ordenado.

Cambiar la clave = eliminar + insertar

Expone: Luis

MODIFICACIÓN: CÓDIGO

ArbolEstudiantes.java

```java
public boolean modificar(int matricula, String cedula,
                         String nombres, double promedio) {
    NodoEstudiante nodo = buscarNodo(raiz, matricula);
    if (nodo == null) {
        return false;                          // no existe
    }
    // primero validar con un Estudiante temporal (lanza excepcion)
    new Estudiante(matricula, cedula, nombres, promedio);   // valida
    // solo despues cambiar los datos del estudiante real
    Estudiante e = nodo.getDato();
    e.setCedula(cedula);
    e.setNombres(nombres);
    e.setPromedio(promedio);
    return true;                               // la matricula NO cambia
}
```

Opción 11 del menú · Complejidad O(h) por buscarNodo

QUÉ MIRAR

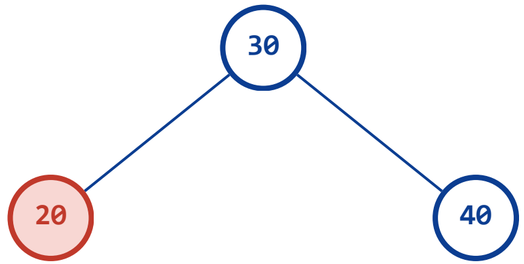
- Busca el nodo (recursivo).
- Si no existe: false.
- Valida primero: si falla, nada cambia.
- Los setters no tocan la matrícula.

Expone: Luis

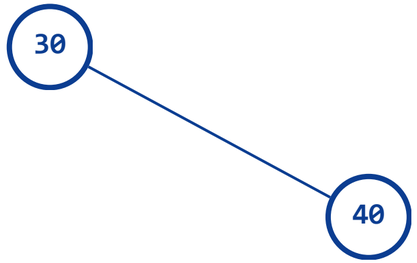
ELIMINACIÓN: 3 CASOS

1 · HOJA (20)

ANTES



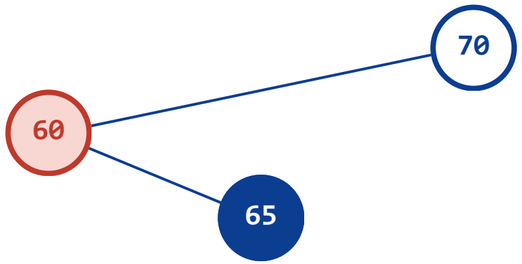
DESPUÉS



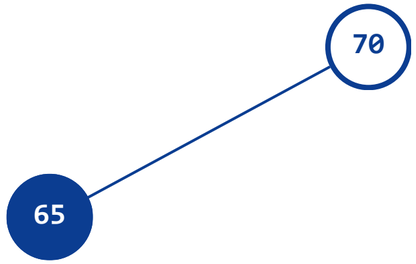
Se elimina sin más: el padre apunta a null.

2 · UN HIJO (60)

ANTES



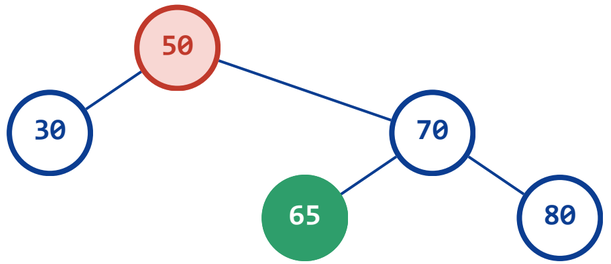
DESPUÉS



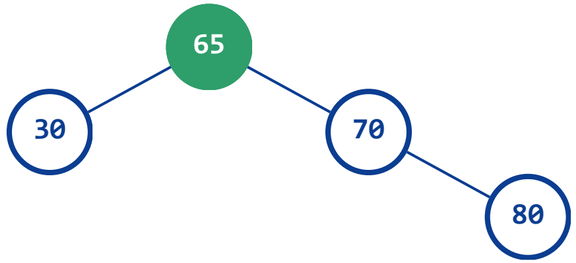
El hijo ocupa el lugar del nodo eliminado.

3 · DOS HIJOS (50)

ANTES



DESPUÉS



Lo reemplaza el sucesor inorden: el menor de la derecha.

Expone: Luis

ELIMINACIÓN: CÓDIGO

ArbolEstudiantes.java

```java
private NodoEstudiante eliminar(NodoEstudiante nodo, int matricula) {
    if (nodo == null) { return nodo; }                // caso base: no se encontro
    int actual = nodo.getDato().getMatricula();
    if (matricula < actual) {
        nodo.setIzquierdo(eliminar(nodo.getIzquierdo(), matricula));
    } else if (matricula > actual) {
        nodo.setDerecho(eliminar(nodo.getDerecho(), matricula));
    } else {                                           // lo encontre
        if (nodo.getIzquierdo() == null && nodo.getDerecho() == null) {
            return null;                               // caso 1: hoja
        }
        if (nodo.getIzquierdo() == null) { return nodo.getDerecho(); }   // caso 2
        if (nodo.getDerecho() == null) { return nodo.getIzquierdo(); }   // caso 2
        Estudiante sucesor = minimo(nodo.getDerecho());  // caso 3: sucesor inorden
        nodo.setDato(sucesor);
        nodo.setDerecho(eliminar(nodo.getDerecho(), sucesor.getMatricula()));
    }
    return nodo;
}
```

QUÉ MIRAR

- Baja hasta encontrar la matrícula.
- Hoja y un hijo: se devuelve el hijo (o null).
- Dos hijos: copia el sucesor y lo borra de la derecha.
- Costo O(h).

Expone: Luis

COMPLEJIDAD

Operación	Árbol general	ABB	Por qué
Buscar	O(n)	**O(h)**	ABB descarta un lado en cada paso
Agregar / insertar	O(n)	**O(h)**	Primero se busca el padre o el lugar
Recorrer	O(n)	**O(n)**	Cada nodo se visita una vez
Modificar	NO APLICA	**O(h)**	Solo la búsqueda; los setters son O(1)
Eliminar	-	**O(h)**	Buscar + el menor de la derecha

Balanceado: h ≈ log n
con 1000 estudiantes, unas 10 comparaciones

Ordenado: h = n (cadena)
el ABB se comporta como una lista: O(n)

Expone: Damian

RETO · RECORRIDOS EN SW-N3-A

Paralelo SW-N3-A vacío. Se matriculan, en este orden:

`45` `25` `70` `10` `30` `60` `90` `27` `65`

1. Dibuja el ABB que resulta.
2. Predice preorden, inorden y posorden.
3. Completa los 3 métodos recursivos con traza.
4. Compara y prueba los casos límite.

Casos límite: árbol vacío · un solo nodo · cadena 10, 20, 30, 40 · repetido

Expone: Mónica

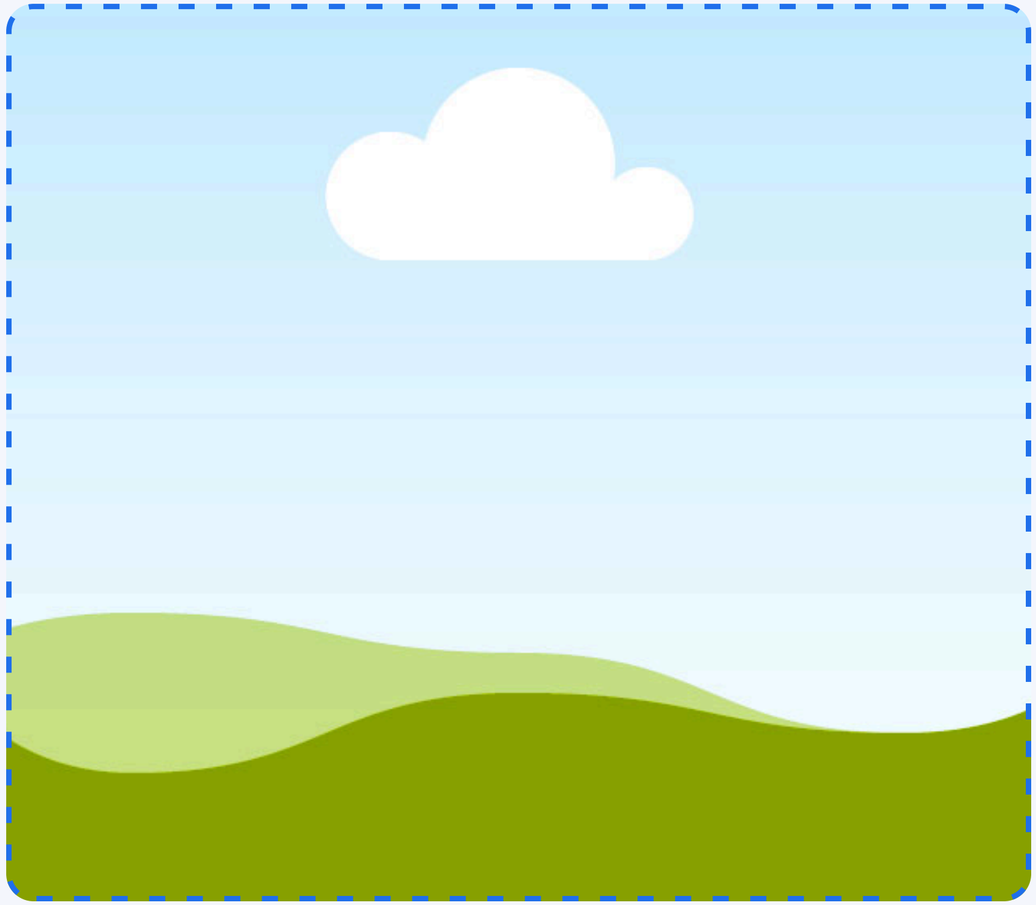
REGLAS

- Solo recursión.
- Caso base: nodo == null.
- Una llamada por cada hijo.
- Traza: entra · null -> regresa · VISITA

RETO · 20 MIN

3 min: presentación
14 min: parejas (4 dibujar, 7 codificar, 3 comparar)
3 min: solución

QUIZ DE REPASO



10 preguntas sobre fundamentos de árboles

3 — **de concepto**

Raíz, condiciones de un árbol binario y nodos hermanos.

2 — **de lectura de código**

Referencias a hijos, grado y tipo Nodo<T>.

5 — **de aplicación**

Árbol general, niveles, altura y subárboles.

Después de cada pregunta explicaremos brevemente la respuesta correcta.

Expone: Jampier

CIERRE

1 Caso base

`nodo == null`

2 Una llamada

`por cada hijo`

3 Costos

O(h) en el ABB · O(n) recorridos

ERRORES COMUNES

- Olvidar el caso base: la recursión no termina.
- No enlazar el resultado al regresar (setIzquierdo).
- Eliminar sin separar los 3 casos.
- Cambiar la matrícula y romper el orden.

EVIDENCIA

- Repositorio de GitHub y Moodle:
- diapositivas, código y trazas,
- enlace del quiz y banco de preguntas,
- README y diagramas de secuencia y flujo.

Lunes 19/10 · Fase 2: mostrarJerarquía, altura, grado, nivel y contarHojas

Expone: Jampier · Mónica · Damian

MUCHAS GRACIAS